

Piattaforma cloud-native a microservizi per Aspect-Based Sentiment Analysis di recensioni bancarie

Dario Lazzara

Matricola 1000054995

Anno Accademico 2025/2026

Sommario

Nella presente relazione si descrive la progettazione e la realizzazione di una piattaforma cloud per l'analisi del sentiment orientata agli aspetti (ABSA) su recensioni bancarie. L'applicazione ha un'architettura a tre livelli basata su microservizi: un frontend in Streamlit, un API gateway in FastAPI, un worker di inferenza che esegue un modello LCF-ATEPC costruito su BERT e un database PostgreSQL. Dato che l'inferenza su CPU è piuttosto lenta, si è scelto di disaccoppiare il percorso sincrono con una coda di messaggi in tal modo l'API risponde subito con **202 Accepted**, mentre i worker elaborano i task al proprio ritmo usando micro-batching per questioni di efficienza. Il numero di worker viene regolato automaticamente da KEDA in base alla lunghezza della coda, permettendo di arrivare fino a zero repliche quando la coda resta vuota per un certo intervallo di tempo. Il progetto è stato sviluppato in due fasi: prima un cluster Kubernetes locale su OrbStack, poi lo stesso sistema portato su AWS con EKS, RDS, SQS, S3 ed ECR. Tutta l'infrastruttura è descritta come codice con Terraform e Ansible, e una pipeline su GitHub Actions esegue lint, test e controlli Terraform e pubblica le immagini su ECR autenticandosi via OIDC, senza credenziali statiche. Nel testo vengono motivate le scelte progettuali alla luce dei vincoli reali tra cui il budget limitato e l'impossibilità di sfruttare una GPU per l'inferenza.

Indice

1	Introduzione	3
2	Il caso d'uso: ABSA di recensioni bancarie	3
2.1	Aspect-Based Sentiment Analysis	3
2.2	Il modello utilizzato	4
2.3	Perché l'architettura deve essere asincrona	4
3	Architettura del sistema	4
3.1	Vista d'insieme	4
3.2	Decisioni di progetto	7
4	Fase A: deployment locale su OrbStack	8
4.1	Docker Compose	9
4.2	Kubernetes su OrbStack	9
4.3	Autoscaling con KEDA	10
4.4	IaC locale: Terraform e Ansible	10
5	Fase B: deployment su AWS	11
5.1	Due stack Terraform: <code>persistent</code> e <code>compute</code>	11
5.2	Rete	13
5.3	Cluster EKS	13
5.4	Tier dati: RDS	14
5.5	Coda: SQS con dead letter queue	14
5.6	Il modello da S3	14
5.7	Identità e permessi: IRSA	14
5.8	Autoscaling su SQS: cooldown e cold start	15
5.9	Deploy con Ansible e rendering dei manifest	15
5.10	Costi e strategia di teardown	16
6	CI/CD	17
7	Test e risultati	18
7.1	Metodologia	18
7.2	Risultati	19
8	Howto: riprodurre l'infrastruttura	20
8.1	Prerequisiti	20
8.2	Fase A (locale)	20
8.3	Fase B (AWS)	21
8.4	Teardown	22
9	Il repository	23
10	Conclusioni	23
	Riferimenti	25
	Elenco delle figure	26
	Elenco delle tabelle	26

1 Introduzione

Il progetto realizza una piattaforma per l'analisi automatica di recensioni bancarie raccolte da Trustpilot: l'utente invia una recensione in italiano e il sistema ne estrae gli *aspetti* citati (per esempio l'app, i costi, l'assistenza), assegnando a ciascuno una polarità positiva, negativa o neutra. I risultati finiscono in una dashboard che mostra, banca per banca, come si distribuisce il sentiment sui vari aspetti del servizio. Il modello di analisi impiegato è un LCF-ATEPC il quale è stato sottoposto a fine-tuning in un progetto precedente (absa-trustpilot-banking). Il presente elaborato si concentra invece sulla progettazione e sull'implementazione dell'*architettura cloud* necessaria a servire tale modello in modo scalabile e resiliente. A tal fine, si è definita un'architettura a tre livelli basata su microservizi [7], containerizzata con Docker e orchestrata tramite Kubernetes. Per garantire robustezza e alte prestazioni, la persistenza dei dati è stata affidata a un database relazionale, mentre la comunicazione tra i servizi è gestita in modo asincrono tramite una coda di messaggi. L'intera infrastruttura è stata inoltre concepita seguendo i classici paradigmi: definizione *as-code* (IaC) mediante Terraform [11] e Ansible, automazione dei processi di rilascio con pipeline CI/CD su GitHub Actions e, infine, migrazione verso un ambiente di produzione su servizi gestiti da AWS.

Il lavoro è stato organizzato in due fasi. Nella **Fase A**, quella locale, tutti i servizi girano in locale sulla macchina (macOS con OrbStack): prima con Docker Compose per iterare velocemente, poi su un cluster Kubernetes integrato in OrbStack, con autoscaling KEDA pilotato dalla coda RabbitMQ. Questa fase non costa nulla ed è risultata utile principalmente per mettere a punto l'intera architettura applicativa prima di spostarsi sul cloud. Nella **Fase B** la stessa applicazione viene migrata su servizi gestiti da AWS: EKS per l'orchestrazione, RDS per PostgreSQL, SQS come coda, S3 per i pesi del modello ed ECR come registry delle immagini. Il codice applicativo resta identico, cambia solo la configurazione su cui gira.

Il vincolo che ha guidato più di ogni altro le decisioni progettuali è il budget disponibile, da cui discendono l'uso di istanze spot per i nodi, un solo NAT gateway, RDS nella configurazione minima e soprattutto la scelta di separare l'infrastruttura in una parte persistente, ed una parte di calcolo che viene distrutta alla fine di ogni sessione di lavoro (sezione 5). Per quanto concerne il problema dell'utilizzo esclusivo della CPU per l'inferenza, esso è stato assorbito nell'architettura, tramite il disaccoppiamento asincrono, il micro-batching e lo scaling orizzontale dei worker.

A questo si affianca un requisito metodologico seguito in tutto il progetto, l'infrastruttura interamente *as-code*: non è stata creata nessuna risorsa dalla console web di AWS, tutto passa da Terraform e Ansible, e la CLI di AWS viene usata solo per ispezionare e verificare quello che è stato creato.

Il filo conduttore prosegue così: il caso d'uso e il modello (sezione 2), l'architettura e le decisioni di progetto (sezione 3), le due fasi di deployment (sezioni 4 e 5), la pipeline CI/CD (sezione 6), la metodologia di test (sezione 7), una guida passo passo per riprodurre l'infrastruttura (sezione 8) e infine la struttura del repository (sezione 9).

2 Il caso d'uso: ABSA di recensioni bancarie

2.1 Aspect-Based Sentiment Analysis

La sentiment analysis classica assegna una polarità complessiva a un testo, ma per le recensioni di servizi bancari questa granularità non basta: una recensione come «l'app è comodissima ma l'assistenza è pessima» non è né positiva né negativa nel suo insieme, è positiva sull'aspetto *app* e negativa sull'aspetto *assistenza*. L'Aspect-Based Sentiment Analysis (ABSA) [1] lavora proprio a questo livello di dettaglio: individua i termini di aspetto presenti nel testo e assegna una polarità a ciascuno. Per chi deve analizzare il servizio di una banca fa una bella differenza sapere che «i clienti sono scontenti» rispetto a sapere che «i clienti apprezzano l'app ma lamentano i tempi dei bonifici».

2.2 Il modello utilizzato

Il modello proviene dal progetto `absa-trustpilot-banking`, dove era stato fine-tuned su recensioni bancarie estratte da Trustpilot. Le sue caratteristiche principali, ai fini di questo lavoro, sono:

- architettura **LCF-ATEPC** [2], che estrae i termini di aspetto e ne classifica la polarità in un unico passaggio;
- base pre-addestrata `dbmdz/bert-base-italian-xxl-cased` [5], un modello BERT [4] per l'italiano, servita tramite la libreria PyABSA ($\geq 2.4.2$) [3];
- checkpoint addestrato di circa 450 MB (i quattro file `lcf_atepc.config`, `.state_dict`, `.tokenizer`, `.args.txt`);
- l'output è fatto di *coppie* termine di aspetto/polarità, con la relativa confidenza.

L'interfaccia di inferenza è quella nativa di PyABSA: si istanzia un `AspectExtractor` puntando alla cartella del checkpoint e si invoca `extract_aspect` passando una *lista* di testi.

Una delle prime decisioni progettuali adottate è stata di **non inserire nell'immagine Docker** il checkpoint del modello. Il checkpoint viene fornito al worker dall'esterno: in locale è montato come volume, in cloud viene scaricato da S3 all'avvio del pod da un *initContainer* dedicato (sezione 5.6).

Le ragioni sono due. Un'immagine con dentro 450 MB di pesi sarebbe lenta da buildare, da pushare e da scaricare a ogni scalata del worker. Soprattutto, legherebbe il ciclo di vita dell'immagine, che cambia insieme al codice, a quello del modello, che invece non cambia praticamente mai.

2.3 Perché l'architettura deve essere asincrona

L'inferenza gira su CPU, e su CPU un forward di BERT-base richiede tempi dell'ordine del secondo per ogni recensione. Un'API sincrona che eseguisse il modello dentro il ciclo richiesta-risposta comporterebbe due problemi seri: latenze inaccettabili per il client e, appena arriva un picco di traffico, la saturazione immediata del servizio, perché la capacità di ingestione finirebbe per coincidere con la capacità di inferenza.

La coda di messaggi rompe proprio questo accoppiamento. L'API si limita a validare la richiesta, salvarla e accodare un task: operazioni che costano pochi millisecondi, quindi l'ingestione regge picchi anche molto alti. I worker consumano la coda al ritmo che la CPU gli permette, e la coda stessa fa da cuscinetto tra i due ritmi. Da qui viene la proprietà che rende il sistema elastico: la lunghezza della coda è una misura diretta del lavoro arretrato, ed è quindi la metrica naturale su cui basare lo scaling orizzontale dei worker. Scalare i worker vuol dire scalare la capacità di inferenza, in modo lineare e senza toccare nient'altro nel sistema.

3 Architettura del sistema

3.1 Vista d'insieme

L'applicazione è composta da quattro componenti applicativi e due infrastrutturali, distribuiti sui tre tier canonici (tabella 1); la figura 1 ne mostra la disposizione e i flussi nell'ambiente locale.

Il percorso di una recensione nel sistema si svolge in due tempi, separati dalla coda (figura 2). Nella fase di ingestione il frontend invia una `POST /reviews` all'API. L'API genera un identificatore UUID, inserisce la recensione su PostgreSQL con stato `pending`, pubblica in coda un messaggio che contiene il solo `review_id` e risponde `202 Accepted` con l'identificatore (codice 1). Sono tutte operazioni da pochi millisecondi così il client non attende l'inferenza.

Tabella 1: Componenti della piattaforma e loro collocazione nei tier.

Tier	Componente	Ruolo
1 — Presentazione	Frontend (Streamlit)	Invio recensioni con polling del risultato; dashboard del sentiment aggregato per aspetto.
2 — Logica	API gateway (FastAPI)	Riceve le recensioni via HTTP, le persiste come pending , pubblica un task in coda e risponde 202 Accepted ; espone le letture per la dashboard.
2 — Logica	Inference worker (Python + PyABSA)	Consuma i task dalla coda in micro-batch, esegue il modello ABSA e salva gli aspetti estratti. È il componente replicato dall'autoscaler.
3 — Dati	PostgreSQL	Recensioni, stato di elaborazione e aspetti estratti; sorgente unica per dashboard e statistiche.
Async	Coda di messaggi	RabbitMQ in locale, Amazon SQS in cloud; disaccoppia API e worker.
Async	KEDA [10]	Autoscaler event-driven: scala i worker $0 \rightarrow N$ sulla lunghezza della coda.

In questa fase, si evidenziano due scelte progettuali fondamentali per garantire la robustezza del sistema:

- **Messaggi leggeri (Claim Check Pattern [6]):** Il messaggio inviato alla coda non contiene il testo della recensione, ma esclusivamente il suo identificativo (`review_id`). In questo modo, i messaggi restano di dimensioni minime e il database funge da unica fonte di verità. Questo approccio previene inconsistenze e rende innocua un'eventuale riconsegna del messaggio da parte del broker.
- **Pubblicazione dopo il commit:** La recensione viene prima salvata nel database e solo dopo il commit il messaggio viene pubblicato in coda; pubblicando prima, un worker libero potrebbe ricevere il messaggio quando la recensione non è ancora visibile e scartarlo. Se la coda risulta irraggiungibile, la recensione appena inserita viene rimossa e il client riceve un errore 503. Tale meccanismo scongiura il rischio che una recensione resti nel database in stato **pending** senza essere mai accodata per l'analisi.

L'elaborazione avviene dall'altra parte della coda. Il worker preleva i messaggi a blocchi (sezione 3.2), legge i testi dal database portando le recensioni in stato **processing**, esegue l'inferenza in batch e salva gli aspetti estratti, chiudendo le recensioni in stato **done** (o **error** in caso di problemi). Solo a quel punto i messaggi vengono confermati alla coda, con un `ack` su RabbitMQ o una `delete` su SQS.

Il risultato torna al client per polling: il frontend interroga periodicamente `GET /reviews/{id}` e mostra gli aspetti appena lo stato diventa **done**. La dashboard segue invece un percorso indipendente e usa `GET /stats`, che aggrega per aspetto e polarità direttamente in SQL.

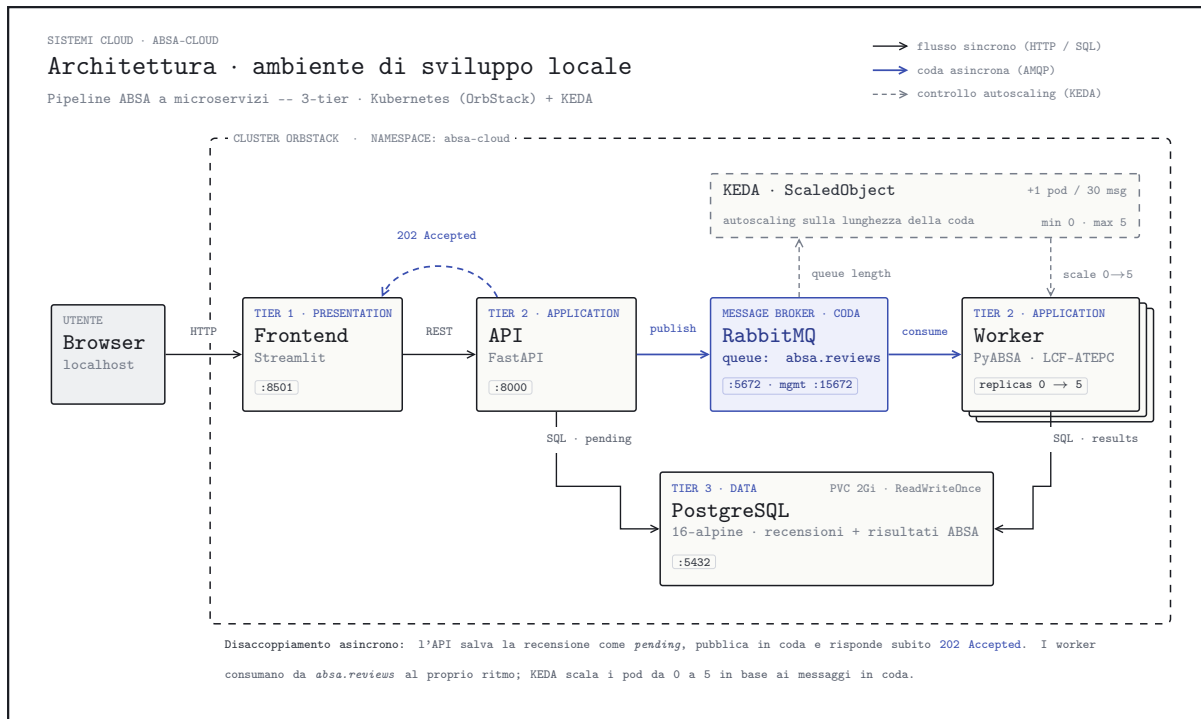


Figura 1: Architettura della Fase A (deployment locale su OrbStack). Il flusso sincrono si ferma all'API, che risponde **202 Accepted**; oltre la coda l'elaborazione procede in modo indipendente, e KEDA regola il numero di worker sulla profondità di *absa.reviews*.

Codice 1: L'endpoint di ingestione: persistenza, pubblicazione in coda e risposta immediata (estratto da `services/api/app/main.py`).

```
@app.post("/reviews", status_code=202, response_model=ReviewAccepted)
def create_review(review: ReviewIn):
    review_id = str(uuid.uuid4())
    # prima il commit, poi la coda
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO reviews (id, bank, text, status) "
                "VALUES (%s, %s, %s, 'pending')",
                (review_id, review.bank, review.text),
            )

    try:
        publish({"review_id": review_id})
    except Exception:
        # senza messaggio in coda nessuno la elaborerebbe
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("DELETE FROM reviews WHERE id = %s", (review_id,))
            raise HTTPException(status_code=503, detail="Queue unavailable") from None
    return {"id": review_id, "status": "pending"}
```

Lo schema dati è stato mantenuto volutamente semplice: una tabella **reviews** (identificatore, banca, testo, stato, timestamp di creazione ed elaborazione) e una tabella **aspects** (aspetto, polarità, confidenza) in relazione uno-a-molti con la prima, con gli indici che servono alle letture della dashboard.

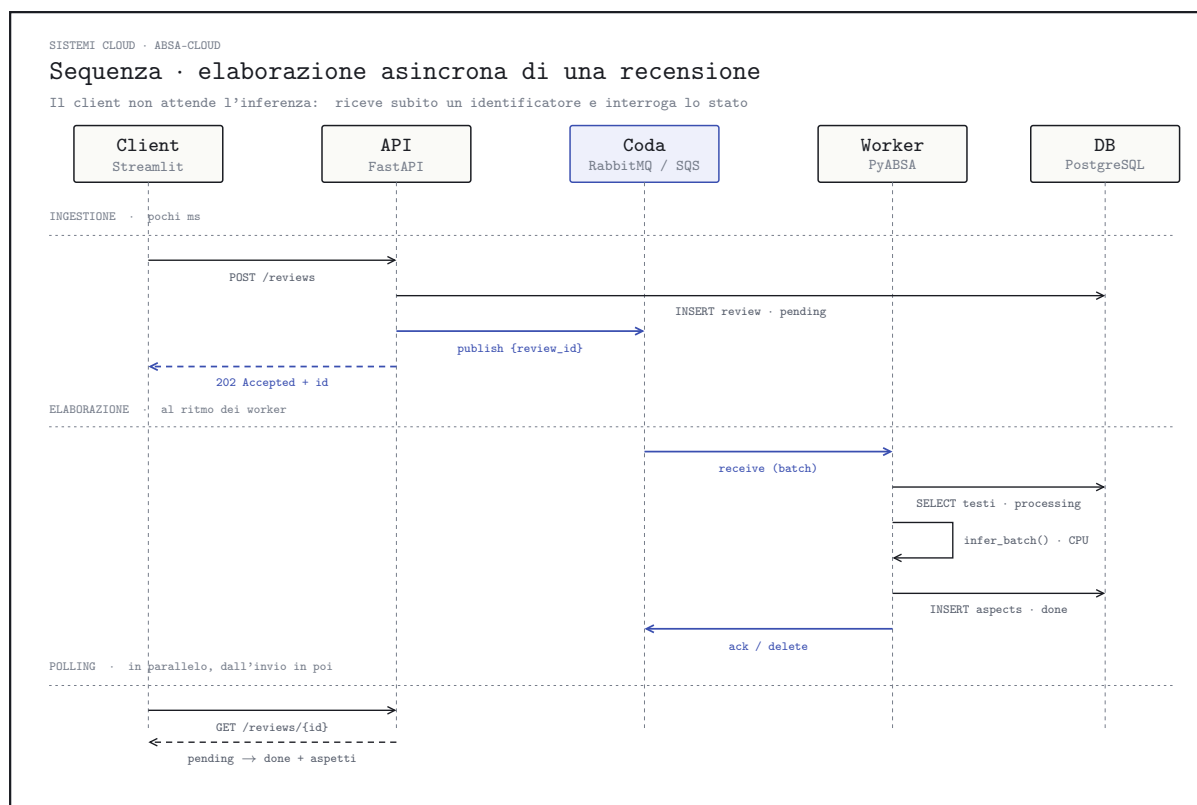


Figura 2: Sequenza di elaborazione asincrona di una recensione. Le tre fasi sono indipendenti: l'ingestione dura pochi millisecondi, l'elaborazione procede al ritmo dei worker e il polling del client corre in parallelo a entrambe.

3.2 Decisioni di progetto

Una recensione, un messaggio. È stato scelto di mettere in coda una recensione alla volta. L'alternativa, accodare blocchi di recensioni, avrebbe ridotto il numero di messaggi ma peggiorato il resto: il singolo messaggio è l'unità naturale di retry (se un'elaborazione fallisce si riconsegna una recensione, non un blocco arbitrario di recensioni miste), lo stato per ogni recensione resta tracciabile, e la lunghezza della coda mantiene un significato diretto, cioè quante recensioni sono in attesa, che è esattamente la metrica usata per lo scaling.

Micro-batching lato worker. Se l'ingestione lavora a grana singola, l'inferenza no. Un modello BERT è più efficiente quando elabora più testi in un'unica chiamata, perché ammortizza l'overhead di tokenizzazione e sfrutta il parallelismo delle librerie di calcolo. Per questo il worker accumula messaggi fino a `BATCH_SIZE` (16 in locale) e invoca `infer_batch` una sola volta per l'intero blocco. Su RabbitMQ, per accumulare i messaggi in modo efficiente, il worker sfrutta il meccanismo di prefetch (`basic_qos`): richiede esplicitamente al broker un blocco di messaggi di dimensione massima pari a quella del batch. A questo meccanismo è affiancato un timeout di inattività (`FLUSH_TIMEOUT`, pari a 2 secondi): qualora non arrivassero nuovi messaggi entro questo limite temporale, il worker procede comunque all'elaborazione del blocco parziale ricevuto fino a quel momento. Ciò assicura che anche una singola recensione isolata venga analizzata prontamente, senza restare in attesa indefinita. Su SQS l'equivalente è una `receive_message` con `MaxNumberOfMessages` pari al batch (con il limite di 10 imposto dal servizio) e long polling di 20 secondi, che riduce sia le richieste a vuoto sia il loro costo.

Conferma dei messaggi e gestione degli errori. La logica di elaborazione (`process_batch`) è indipendente dal backend di coda: riceve coppie `handle/review_id` e restituisce gli `handle` dei messaggi conclusi, delegando al chiamante la conferma tramite il comando specifico del broker (`ack` o `delete`). La conferma viene inviata esclusivamente dopo il commit dei risultati sul database; in questo modo, in caso di terminazione inattesa di un worker a metà batch, nessun dato viene perso. I messaggi non confermati vengono automaticamente riconsegnati (su SQS, allo scadere del *visibility timeout*). Poiché una riconsegna potrebbe rielaborare recensioni già processate, l'operazione di salvataggio è stata resa idempotente: gli eventuali aspetti già estratti per quella specifica recensione vengono rimossi e riscritti da zero.

Infine, la gestione degli errori differenzia tre casistiche in base alla natura del fallimento:

- **Messaggi malformati o non validi** (es. recensione inesistente): vengono confermati e scartati immediatamente. Ritentare l'elaborazione sarebbe inutile e genererebbe un *poison message* in grado di bloccare la coda.
- **Errori di elaborazione** (es. fallimento del modello): la recensione viene marcata con lo stato `error` nel database e il messaggio viene comunque confermato per rimuoverlo dalla coda.
- **Errori infrastrutturali** (es. database irraggiungibile): i messaggi vengono lasciati intenzionalmente come non confermati, delegando il *retry* al broker e, in ambiente cloud, all'inoltro verso una *Dead Letter Queue* dopo cinque tentativi falliti (sezione 5.5).

Due modalità di inferenza. Il motore di inferenza ha due modalità, selezionate dalla variabile `MODEL_MODE`. La modalità `mock` usa un'euristica a parole chiave in italiano: riconosce sette aspetti ricorrenti nel dominio bancario e stima la polarità dai termini vicini all'aspetto. Non ha nessuna pretesa di qualità, serve solo per sviluppare e testare l'architettura, cioè codice, scaling e deployment, senza dipendenze pesanti, senza checkpoint e con immagini Docker leggere. La modalità `pyabsa` carica invece il modello vero: richiede l'immagine buildata con `--build-arg INSTALL_ML=true` (che aggiunge PyABSA e PyTorch, portando l'immagine a circa 3 GB) e il checkpoint raggiungibile al path indicato da `ABSA_CHECKPOINT`. Se il caricamento del modello fallisce, il worker torna al `mock` invece di andare in crash loop. I wheel di PyTorch usati sono quelli `+cpu`, senza librerie CUDA, in quanto sui nostri nodi solo CPU sarebbero gigabyte sprecati nell'immagine.

Il backend di coda dietro una variabile. Per portare il codice tra le due fasi senza riscriverlo si è usato un'astrazione minima: la variabile `QUEUE_BACKEND` seleziona RabbitMQ (`rabbitmq`, il default) o SQS (`sqs`) sia nel publisher dell'API sia nel consumer del worker. L'interfaccia vista dal resto del codice è identica in entrambi i casi, quindi passare dal locale al cloud non tocca una riga di logica applicativa. Si è scartata l'alternativa di usare lo stesso broker in entrambe le fasi, per esempio RabbitMQ gestito anche in cloud, per le ragioni spiegate nella sezione 5.5.

Configurazione a dodici fattori. Tutta la configurazione passa per variabili d'ambiente (tabella 2), iniettate da Compose in locale e da ConfigMap e Secret su Kubernetes. Le stesse immagini girano senza modifiche in ogni ambiente.

4 Fase A: deployment locale su OrbStack

La Fase A si articola in due livelli di maturità: Docker Compose per lo sviluppo rapido quotidiano, e un cluster Kubernetes locale per validare orchestrazione e autoscaling. Per l'implementazione del cluster locale su architettura Apple Silicon, in alternativa a soluzioni basate su macchine virtuali tradizionali (come Multipass), si è optato per il Kubernetes nativo integrato in

Tabella 2: Principali variabili d'ambiente della piattaforma.

Variabile	Componenti	Significato
QUEUE_BACKEND	api, worker	Backend di coda: <code>rabbitmq</code> (default) o <code>sqs</code> .
QUEUE_NAME	api, worker	Nome della coda RabbitMQ (<code>absa.reviews</code>).
RABBITMQ_URL	api, worker	URL AMQP del broker (Fase A).
SQS_QUEUE_URL	api, worker	URL della coda SQS (Fase B).
AWS_REGION	api, worker	Regione AWS per i client boto3.
DATABASE_URL	api, worker	Stringa di connessione PostgreSQL (locale o RDS).
MODEL_MODE	worker	<code>mock</code> (euristica) o <code>pyabsa</code> (modello reale).
ABSA_CHECKPOINT	worker	Path della cartella-checkpoint LCF-ATEPC.
MOCK_DELAY	worker	Secondi di attesa per batch in modalità <code>mock</code> , per simulare il costo dell'inferenza (0 di default, 2 su Kubernetes locale).
BATCH_SIZE	worker	Dimensione del micro-batch (16 in locale, 10 su SQS).
FLUSH_TIMEOUT	worker	Secondi di inattività prima di elaborare un batch parziale (RabbitMQ).
SQS_VISIBILITY_TIMEOUT	worker	Visibilità dei messaggi in elaborazione (300 s).
SQS_ERROR_BACKOFF	worker	Secondi di attesa prima di riprovare se la lettura da SQS fallisce (5 s).
MODEL_BUCKET	worker (init)	Bucket S3 da cui scaricare il checkpoint (Fase B).
API_URL	frontend	Endpoint dell'API gateway.

OrbStack. Questa scelta garantisce le funzionalità di un cluster reale, abbattendo al contempo l'overhead di un ulteriore strato di virtualizzazione.

4.1 Docker Compose

Ogni microservizio ha il proprio Dockerfile (base `python:3.11-slim`); il file `docker-compose.yml` orchestra i cinque servizi, PostgreSQL, RabbitMQ con l'interfaccia di management, API, worker e frontend, con healthcheck e dipendenze ordinate: l'API parte solo quando database e broker risultano sani. Lo schema SQL viene caricato in automatico al primo avvio di PostgreSQL. Tutto l'ambiente si esegue con:

```
docker compose up --build
# frontend http://localhost:8501
# API      http://localhost:8000/docs
# RabbitMQ http://localhost:15672 (guest/guest)
```

Docker Compose consente inoltre di effettuare un primo test di *scaling* orizzontale manuale (tramite il flag `--scale worker=3`), utile per validare la corretta distribuzione dei task su più repliche dello stesso worker concorrenti sulla medesima coda. Tuttavia, non offrendo alcun meccanismo di *autoscaling* dinamico, tale strumento risulta limitato; per colmare questa lacuna e adattare il numero di worker al traffico reale è stato necessario l'impiego di Kubernetes [9], che nasce proprio dall'esperienza di Google con la schedulazione di container su larga scala [8].

4.2 Kubernetes su OrbStack

I manifest in `k8s/local/` definiscono, nel namespace `absa-cloud`, le Deployment di PostgreSQL, RabbitMQ, API, worker e frontend, i Service corrispondenti, una ConfigMap (`absa-config`) per la configurazione non sensibile e un Secret (`absa-secret`) per credenziali e stringhe di connessione. API e frontend sono esposti come Service di tipo `LoadBalancer`: OrbStack li pubblica direttamente su `localhost`, il che ha reso superflua la necessità di un ingress controller,

un componente in più da installare e mantenere senza alcun vantaggio reale in un ambiente a nodo singolo.

4.3 Autoscaling con KEDA

Il requisito di elasticità è stato soddisfatto integrando KEDA (Kubernetes Event-Driven Autoscaling). La preferenza accordata a questo strumento rispetto al classico `HorizontalPodAutoscaler` (HPA) basato su CPU è motivata da precise esigenze architetturali. L'HPA scala su una metrica di consumo, la percentuale di CPU dei pod, che per un worker a coda è un segnale tardivo e ambiguo: un worker che sta elaborando un batch è al 100% di CPU sia quando in coda ci sono dieci messaggi sia quando ce ne sono diecimila, quindi la CPU da sola non dice quanto lavoro resta da fare. La lunghezza della coda invece misura esattamente il backlog, che è la grandezza che si vuole davvero controllare. KEDA legge questa metrica direttamente dal broker e, cosa decisiva, sa scalare fino a zero, mentre l'HPA standard non scende mai sotto una replica.

Codice 2: ScaledObject del worker in Fase A (estratto da `k8s/local/05-worker.yaml`).

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: worker-keda-scaler
  namespace: absa-cloud
spec:
  scaleTargetRef:
    name: worker
  pollingInterval: 10
  cooldownPeriod: 30
  minReplicaCount: 0    # scale-to-zero a coda vuota
  maxReplicaCount: 5
  triggers:
    - type: rabbitmq
      metadata:
        queueName: "absa.reviews"
        mode: "QueueLength"
        value: "30"      # circa un pod ogni 30 messaggi in coda
      authenticationRef:
        name: rabbitmq-trigger-auth
```

Il trigger RabbitMQ legge la profondità della coda `absa.reviews` e mantiene circa una replica ogni 30 messaggi, fino a un massimo di 5; l'autenticazione verso il broker passa da una `TriggerAuthentication` che punta alla chiave `RABBITMQ_URL` del Secret, evitando così di duplicare credenziali nei manifest. Nell'ambiente di sviluppo locale, il `cooldownPeriod` è impostato a 30 secondi: l'impiego del modello *mock* annulla infatti i tempi di caricamento, consentendo un rapido *scale-to-zero* utile per la validazione visiva dell'autoscaling. In ambiente cloud, al contrario, tale parametro è stato innalzato a 300 secondi al fine di mitigare l'overhead legato al *cold start* del modello reale, come dettagliato nella sezione 5.8.

4.4 IaC locale: Terraform e Ansible

Anche in ambiente locale l'infrastruttura è stata definita come codice (IaC), mantenendo la medesima separazione dei ruoli adottata in cloud: Terraform per il provisioning e Ansible per la configurazione e il deploy. Lo script Terraform (`terraform/local/`), tramite il provider Kubernetes puntato al contesto `orbstack`, provvede alla creazione del *namespace* del progetto e alla

relativa etichettatura. La sua configurazione risulta volutamente minimale poiché, non essendovi reti o database esterni da istanziare in locale, il suo scopo primario è consolidare il flusso di lavoro che verrà poi replicato in cloud. Il playbook Ansible (`ansible/local/playbook.yml`) ha il compito di orchestrare l'intera catena di automazione: invoca `terraform apply`, effettua la build delle tre immagini container, installa KEDA (applicando il manifest ufficiale *server-side*), rilascia i manifest applicativi e infine resta in attesa che tutti i pod raggiungano lo stato `Ready`. In questo modo, un singolo comando consente il passaggio da un ambiente vuoto all'applicazione pienamente operativa:

```
ansible-playbook ansible/local/playbook.yml
```

A deployment completato, la pipeline risulta pienamente operativa *end-to-end*. La figura 3 mostra la dashboard del sentiment popolata a seguito dell'elaborazione delle recensioni. Il comportamento sotto carico — la coda che assorbe il burst e il suo smaltimento a lotti da parte del worker — è osservabile a runtime dall'interfaccia di management di RabbitMQ; la sua dinamica quantitativa, insieme alla reazione di *scaling* pilotata da KEDA, è misurata e commentata in Fase B (sezione 7, figura 7).

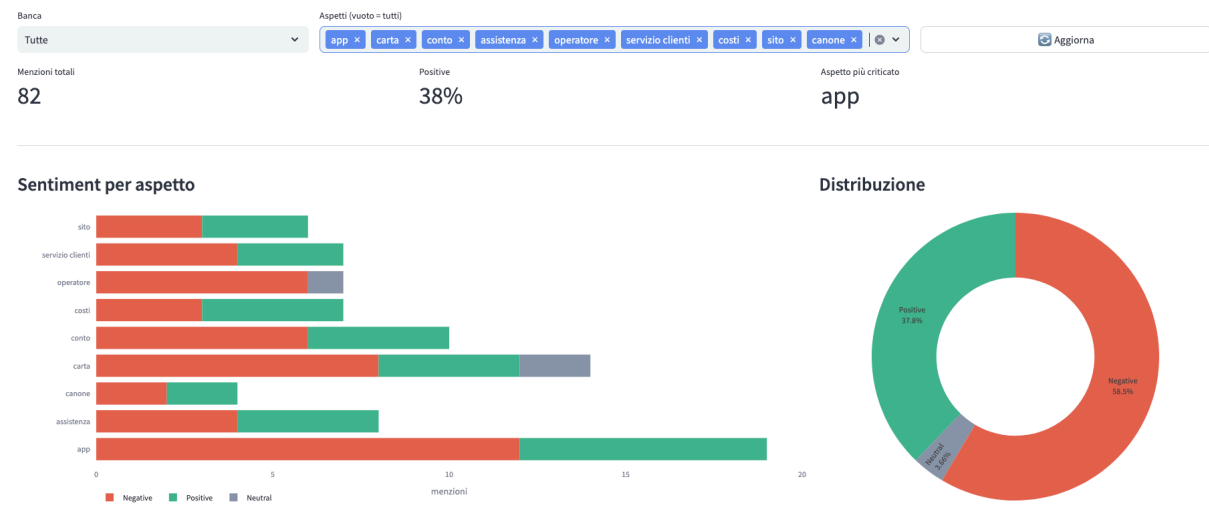


Figura 3: La dashboard del frontend in Fase A, popolata con recensioni reali elaborate dal modello: sentiment per aspetto, distribuzione complessiva e le metriche di sintesi.

5 Fase B: deployment su AWS

Nella Fase B l'infrastruttura è stata interamente migrata verso i servizi gestiti AWS, mantenendo del tutto invariato il codice applicativo. La transizione ha comportato una naturale mappatura delle risorse locali sui corrispondenti servizi cloud: il cluster OrbStack è stato sostituito da Amazon EKS, il database PostgreSQL *in-cluster* è migrato su RDS, RabbitMQ ha ceduto il posto ad SQS, il volume locale del modello è stato convertito in un bucket S3 e, infine, il registry locale è passato su ECR. La figura 4 illustra l'architettura risultante, mentre la tabella 3 ne riassume le componenti.

5.1 Due stack Terraform: *persistent* e *compute*

Una delle decisioni strutturali della Fase B è aver diviso l'infrastruttura in due stack Terraform indipendenti, ciascuno con il proprio stato. Lo stack *persistent* contiene quello che deve

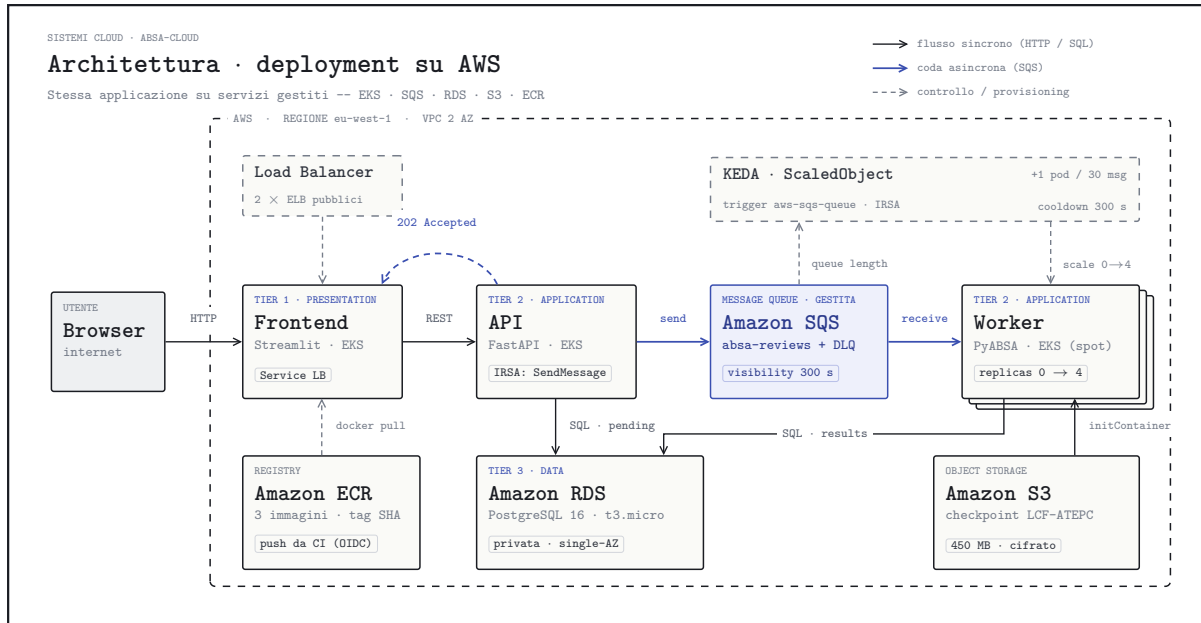


Figura 4: Architettura della Fase B (deployment su AWS). La struttura è la stessa della Fase A: cambiano i servizi che realizzano coda, database e storage del modello, non il codice applicativo.

Tabella 3: Servizi AWS impiegati, loro ruolo e stack Terraform di appartenenza.

Servizio	Ruolo	Stack
S3	Checkpoint del modello (~450 MB), scaricato dai worker all'avvio	persistent
ECR	Registry delle immagini dei tre microservizi	persistent
AWS Budgets	Alert di spesa a 10/50/100 USD	persistent
IAM (OIDC GitHub)	Ruolo assumibile dalla CI per il solo push su ECR	persistent
VPC (2 AZ, 1 NAT)	Rete: subnet pubbliche (ELB, NAT) e private (nodi, RDS)	compute
EKS	Kubernetes gestito; node group spot m7i-flex.large	compute
RDS PostgreSQL	Tier dati gestito (db.t3.micro, single-AZ)	compute
SQS + DLQ	Coda asincrona e dead letter queue	compute
IAM (IRSA)	Ruoli per pod API, worker e KEDA, senza chiavi statiche	compute
KEDA (Helm)	Autoscaling del worker sulla lunghezza della coda SQS	compute

sopravvivere tra una sessione e l'altra e costa cifre trascurabili: il bucket S3 con il modello, i repository ECR con le immagini, gli alert di AWS Budgets e il ruolo OIDC per la CI. Lo stack **compute** contiene tutto il resto, VPC, EKS, RDS, SQS, i ruoli IRSA e l'installazione di KEDA, cioè le risorse che generano costi orari veri e propri.

Il motivo è ovviamente il budget, ma la divisione in due stack lo trasforma anche in una procedura operativa più sicura: a fine sessione basta lanciare `terraform destroy` sul solo stack **compute**, azzerando la spesa di calcolo senza il rischio di cancellare per sbaglio il modello (che andrebbe ricaricato, 450 MB) o le immagini (che andrebbero ribuildate). Con un unico stack sarebbero serviti `destroy` mirati con `-target`, una pratica fragile e che la stessa documentazione Terraform sconsiglia. C'è anche una differenza di fondo nel ciclo di vita in quanto **persistent** si applica una volta e non cambia quasi mai, mentre **compute** nasce e muore con la sessione di lavoro.

5.2 Rete

La VPC (modulo ufficiale `terraform-aws-modules/vpc`) copre due availability zone con una coppia di subnet per AZ (figura 5). Le pubbliche ospitano i load balancer e il NAT gateway, le private i nodi EKS e RDS. Due AZ sono il minimo richiesto sia da EKS sia dai subnet group di RDS. È stato scelto volutamente un solo NAT gateway (`single_nat_gateway = true`). La ridondanza di un NAT per AZ comporterebbe un raddoppio dei costi operativi; trattandosi di un progetto accademico con sessioni di lavoro di breve durata, la protezione dal fallimento di una singola AZ non giustifica tale incremento di spesa. Le subnet portano i tag `kubernetes.io/role/elb` e `kubernetes.io/role/internal-elb`, che servono a EKS per trovare da solo le subnet giuste quando crea i load balancer dei Service.

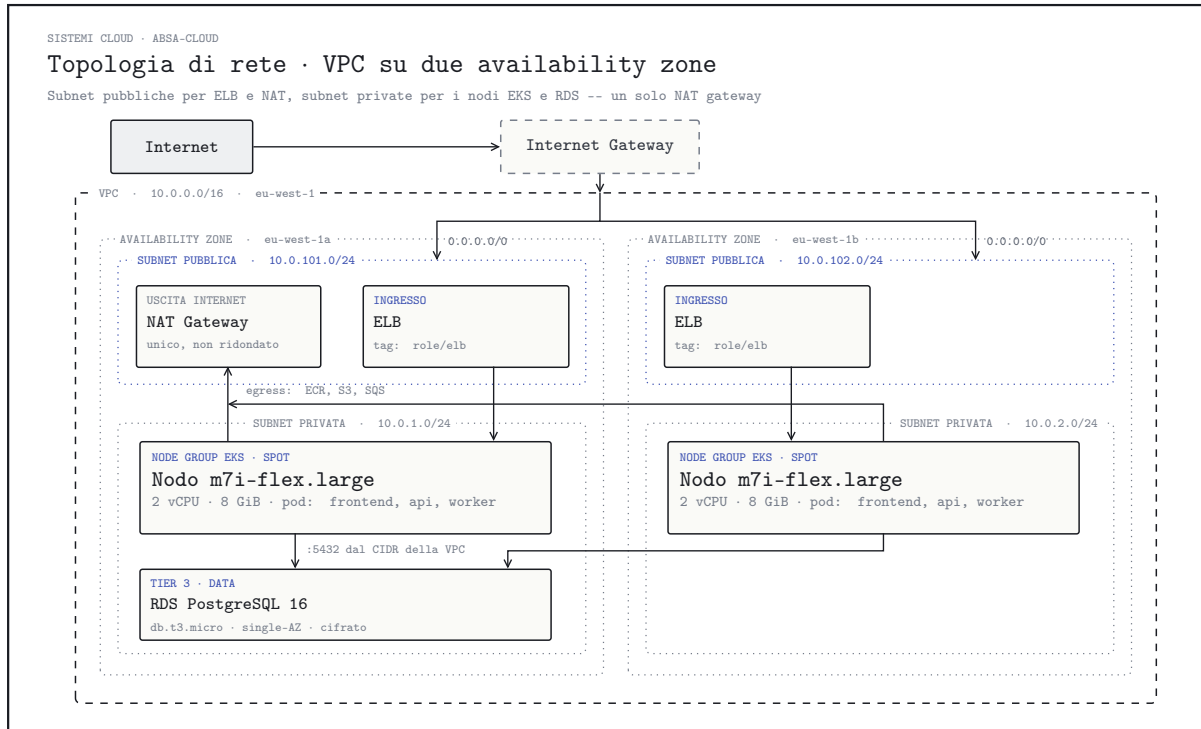


Figura 5: Topologia di rete della Fase B. I nodi EKS e RDS stanno solo in subnet private e raggiungono internet attraverso l'unico NAT gateway; le subnet pubbliche ospitano i load balancer. RDS è single-AZ (l'istanza vive nella sola `eu-west-1a`), ma il DB subnet group richiesto da AWS copre comunque entrambe le subnet private.

5.3 Cluster EKS

Il cluster (modulo `terraform-aws-modules/eks`, Kubernetes 1.34) ha un node group gestito di istanze `m7i-flex.large` (2 vCPU, 8 GiB) in capacità spot, da 1 a 4 nodi con 3 desiderati. Si è dimensionato tutto in base al worker con il modello reale, poiché BERT-base in fp32 più il runtime PyTorch occupano circa 2 GiB per pod. Il limite effettivo per nodo è però la CPU: ogni worker richiede 500m e, insieme ai pod di sistema, su 2 vCPU ne entrano due. Le istanze spot costano una frazione delle on-demand [12], e un'eventuale interruzione non compromette l'architettura in quanto i worker sono stateless, i messaggi non confermati vengono riconsegnati e KEDA rimpiazza subito il pod perso. Il tipo di istanza rispetta anche un vincolo pratico dell'account, che è sul piano Free Tier e rifiuta i tipi non eleggibili. I nodi stanno in subnet private e raggiungono Internet, per esempio per il pull da ECR, passando dal NAT; l'endpoint del control plane invece resta pubblico, così è possibile usare `kubect1` dalla macchina di sviluppo senza dover passare da un bastion host.

5.4 Tier dati: RDS

L'istanza PostgreSQL 16 è ospitata su RDS in classe `db.t3.micro` con 20 GiB di storage cifrato. Il database risiede in una subnet privata ed è isolato da Internet. Il security group dedicato ammette infatti il traffico sulla porta 5432 esclusivamente dal CIDR della VPC, confinando l'accesso ai soli nodi del cluster. Il deployment è stato deliberatamente configurato in modalità *Single-AZ*. L'adozione di un'architettura *Multi-AZ* fornirebbe un livello di disponibilità sovradiimensionato per un ambiente effimero e didattico, comportando un impiego ridondante di risorse non strettamente necessario agli obiettivi del progetto. Lo schema viene inizializzato al deploy da un Job Kubernetes che monta `db/init.sql` da una ConfigMap ed esegue `psql` contro RDS dall'interno della VPC: lo script è scritto in modo idempotente (`CREATE TABLE IF NOT EXISTS`), quindi il Job si può ripetere a ogni deploy senza problemi.

5.5 Coda: SQS con dead letter queue

Per l'ambiente cloud si è optato per Amazon SQS in sostituzione di RabbitMQ. L'adozione di Amazon MQ for RabbitMQ avrebbe minimizzato le differenze rispetto all'ambiente locale, ma è stata scartata in favore di un approccio puramente *serverless*. Mantenere un broker costantemente attivo comporterebbe infatti oneri operativi non in linea con i pattern di traffico previsti, mentre il modello *pay-per-use* di SQS assicura un'elevata efficienza. A ciò si unisce un netto vantaggio in termini di gestione, poiché SQS azzerà le attività di manutenzione dell'infrastruttura sottostante e ne delega la disponibilità al *cloud provider*.

La configurazione prevede due code: quella principale (`absa-reviews`) con visibility timeout di 300 secondi, e una dead letter queue verso cui SQS sposta i messaggi dopo 5 ricezioni fallite. Il visibility timeout è stato dimensionato sull'elaborazione peggiore di un batch, cioè caricamento del modello a freddo più inferenza su CPU, perché un timeout troppo corto farebbe riapparire in coda messaggi ancora in lavorazione, causando elaborazioni duplicate (innocue grazie all'idempotenza, ma comunque uno spreco). La DLQ chiude il cerchio della gestione errori: un messaggio problematico non gira all'infinito, finisce in un posto ispezionabile, con 14 giorni di retention.

5.6 Il modello da S3

Il checkpoint sta in un bucket S3 (cifrato, versionato, con accesso pubblico bloccato), caricato una tantum con `aws s3 sync` dal playbook di deploy. Ogni pod worker ha un initContainer basato sull'immagine `amazon/aws-cli` che, solo se `MODEL_MODE=pyabsa`, sincronizza il checkpoint in un volume `emptyDir` condiviso con il container principale:

Il volume viene montato all'interno della *working directory* (in `/app/models/checkpoint`) per aggirare una limitazione di PyABSA, che fallisce il caricamento se si utilizzano percorsi assoluti esterni. L'initContainer non usa nessuna credenziale: i permessi di lettura sul bucket arrivano dal ruolo IRSA del pod (sezione 5.7).

5.7 Identità e permessi: IRSA

Nessun pod usa chiavi AWS statiche. Con IRSA (IAM Roles for Service Accounts) [13] ogni ServiceAccount è annotato con l'ARN di un ruolo IAM che i pod assumono via OIDC, attraverso il provider creato dal modulo EKS. Si è seguito il principio del minimo privilegio, e ogni trust policy vincola il ruolo al singolo ServiceAccount, namespace incluso, così un ruolo non è assumibile da pod arbitrari del cluster:

- `absa-api`: solo `sqs:SendMessage` (più le letture di metadati della coda), sulla sola coda principale;

Codice 3: L'initContainer che scarica il checkpoint da S3 (estratto da `k8s/aws/04-worker.yaml`).

```
initContainers:
- name: model-download
  image: amazon/aws-cli:2.15.30
  command: ["/bin/sh", "-c"]
  args:
  - |
    set -e
    if [ "$MODEL_MODE" = "pyabsa" ]; then
      aws s3 sync "s3://$MODEL_BUCKET/checkpoint" \
        /models/checkpoint --region "$AWS_REGION";
    else
      echo "MODEL_MODE=$MODEL_MODE: nessun download (mock).";
    fi
  volumeMounts:
  - name: model
    mountPath: /models/checkpoint
```

- **absa-worker**: ricezione e cancellazione dalla coda, più `s3:GetObject` e `s3:ListBucket` sul solo bucket del modello;
- **keda-operator**: solo `sqs:GetQueueAttributes`, quanto basta per leggere la profondità della coda ai fini dello scaling.

I client `boto3` nel codice non ricevono nessuna credenziale esplicita: la catena di risoluzione di default le ottiene dal token proiettato nel pod. È lo stesso principio applicato alla CI (sezione 6), dove il ruolo è vincolato al repository e al branch.

5.8 Autoscaling su SQS: cooldown e cold start

KEDA viene installato dallo stack `compute` tramite il provider Helm, con il ServiceAccount dell'operator annotato con il ruolo IRSA dedicato. Lo `ScaledObject` del worker replica la stessa logica della Fase A, da 0 a 4 repliche, circa una ogni 30 messaggi in coda, con il trigger `aws-sqs-queue` che legge `ApproximateNumberOfMessages`.

La differenza vera rispetto al locale è il `cooldownPeriod`, che è stato portato da 30 a 300 secondi. Il motivo è il costo di avvio di un worker reale: il pull di un'immagine da circa 3 GB, il download del checkpoint da S3 e il caricamento di BERT in memoria costano insieme circa 45 secondi. Con un cooldown breve il sistema sarebbe soggetto a continue oscillazioni, poiché ogni recensione isolata pagherebbe l'intero *cold start*, e un flusso lento ma regolare produrrebbe un continuo ciclo di avvio e spegnimento dei worker. I 5 minuti di coda vuota prima del rientro a zero ammortizzano il cold start sui carichi intermittenti, mantenendo comunque lo *scale-to-zero* quando il sistema è davvero fermo. Questa configurazione riflette il classico *trade-off* tra il contenimento aggressivo dei costi e la latenza percepita dall'utente.

La tolleranza temporale per queste casistiche è stata implementata anche a livello di *frontend*. Nello specifico, il *polling* successivo all'invio di una recensione ammette un'attesa massima di 90 secondi, garantendo così il margine necessario per completare l'intera sequenza di *cold start* nel caso in cui la richiesta intervenga dopo un prolungato periodo di inattività.

5.9 Deploy con Ansible e rendering dei manifest

Il collante tra Terraform e Kubernetes è il playbook `ansible/aws/playbook.yml`. I manifest in `k8s/aws/` sono template con segnaposto `${VAR}` (URL della coda, ARN dei ruoli, URI delle

immagini, stringa di connessione RDS): valori che esistono solo dopo `terraform apply` e che sarebbe scomodo, oltre che rischioso, ricopiare a mano. Il playbook li legge in modo dichiarativo con il modulo `cloud.terraform.terraform_output` da entrambi gli stack, poi:

1. aggiorna il kubeconfig per il cluster EKS;
2. builda e pusha le immagini su ECR con `docker buildx --platform linux/amd64`; il passo si salta con `-e build_images=false` se la CI ha già pubblicato le immagini;
3. carica facoltativamente il checkpoint su S3 (`-e upload_model=true`);
4. renderizza ogni manifest con `scripts/render_manifest.py` e lo applica con `kubectl`;
5. esegue il Job di inizializzazione dello schema su RDS e aspetta i rollout, stampando alla fine gli endpoint pubblici dei load balancer.

Due dettagli della pipeline di *deploy* meritano un approfondimento. L'impiego del flag `--platform linux/amd64` in fase di build risulta strettamente necessario per garantire la compatibilità, dal momento che la macchina di sviluppo si basa su architettura ARM mentre i nodi EKS operano su x86. Inoltre, lo script `render_manifest.py` è incaricato della sostituzione selettiva dei parametri di configurazione nei *manifest*. Tale script interviene unicamente sui segnaposto del tipo `${VAR}`, preservando le variabili di *runtime* `$VAR` destinate agli script interni ai container ed evitandone così una risoluzione prematura.

È stata mantenuta la divisione dei ruoli tra i due strumenti abbastanza classica: Terraform possiede lo stato dell'infrastruttura, Ansible orchestra la procedura di configurazione e deploy, che per natura è imperativa e ordinata.

5.10 Costi e strategia di teardown

La tabella 4 riporta i costi orari dello stack `compute`, ricavati dal billing reale (AWS Cost Explorer). Sul totale si considera anche il traffico dati elaborato dal NAT gateway e alcune voci minori (indirizzi IPv4 pubblici, storage ECR, KMS), tutte dell'ordine di pochi centesimi.

Tabella 4: Costi orari dello stack `compute` (eu-west-1), ricavati dal billing reale di una campagna di test.

Risorsa	Costo orario	Note
Control plane EKS	~0,10 \$	fisso per cluster
3 × <code>m7i-flex.large</code> (spot)	~0,18 \$	~0,06 \$/h ciascuna; on-demand ~0,10 \$/h
NAT gateway	~0,05 \$	più ~0,05 \$/GB elaborato
RDS <code>db.t3.micro</code>	~0,02 \$	più storage (~0,13 \$/GB-mese)
2 × Load Balancer	~0,06 \$	Service api e frontend
IPv4 pubblici	~0,02 \$	0,005 \$/h per indirizzo in uso
SQS, S3, ECR	~0 \$	a consumo, trascurabili a questi volumi
Totale	~0,43 \$	≈ 1,7 \$ per una sessione di 4 ore

A questi ritmi si riesce a coprire con budget economico molte decine di sessioni di lavoro, ma solo a condizione che lo stack `compute` venga distrutto a fine sessione in quanto dimenticarlo acceso costerebbe circa 10 \$ al giorno. Per questo sono state adottate precauzioni specifiche. La procedura di *teardown* è stata incapsulata nello script `scripts/aws_teardown.sh`, che rimuove prima i load balancer creati da Kubernetes e poi invoca `terraform destroy` per lo stack `compute`, semplificandone l'esecuzione. A ulteriore tutela finanziaria, il servizio AWS Budgets è stato configurato per inviare avvisi automatici al superamento delle soglie di 10, 50 e 100 USD. Al contrario, lo stack `persistent` viene mantenuto in esecuzione permanente. L'impatto economico di tale scelta risulta marginale (nell'ordine di pochi centesimi mensili), poiché circoscritto

allo *storage* S3 (450 MB) e al registro ECR, la cui capienza è ottimizzata tramite una *lifecycle policy* che conserva unicamente le ultime 10 immagini.

6 CI/CD

La pipeline (`.github/workflows/ci.yml`) è stata mantenuta volutamente essenziale ed è composta da quattro job: tre controlli in parallelo e la build delle immagini. A ogni push su `main` (o su richiesta manuale via `workflow_dispatch`) il job `lint` esegue `ruff` sul codice Python, il job `terraform` verifica formattazione e validità dei tre stack (`fmt` e `validate`, senza credenziali AWS) e il job `test` esegue, misurando la copertura, i test unitari della cartella `tests/`. Questi coprono le logiche più delicate e verificabili senza infrastruttura, ovvero il contratto di conferma dei messaggi del worker (`process_batch`, con database e inferenza simulati: batch corretti, messaggi orfani, errori del modello, database irraggiungibile), la creazione di una recensione nell'API (pubblicazione dopo il commit e 503 con rimozione della recensione se la coda non risponde), il classificatore mock e il rendering dei manifest (`render_manifest.py`, cioè la distinzione tra segnaposto `${VAR}` da risolvere e variabili runtime `$VAR` da lasciare intatte). Solo se tutti e tre passano, il job `build-and-push` builda le immagini dei tre microservizi e le pubblica su ECR con doppio tag, lo SHA del commit e `latest`. In questo modo, un'immagine con logica di elaborazione non funzionante non raggiunge mai il registry. Il tag per commit dà tracciabilità e permette il rollback, mentre `latest` agevola le procedure di deploy manuale. Esclusivamente per il microservizio worker, il processo di build inietta l'argomento `INSTALL_ML=true`, garantendo l'inclusione delle dipendenze necessarie all'esecuzione del modello. Poiché i runner di GitHub Actions operano nativamente su architettura `x86_64` (coerentemente con i nodi EKS), in questa fase non si rende necessaria la cross-compilazione, altrimenti obbligatoria durante lo sviluppo locale su macchine ARM. La figura 6 mostra un'esecuzione della pipeline conclusa con successo.

La parte a cui si è dedicata più attenzione è l'autenticazione. Il workflow non contiene nessuna chiave AWS: chiede a GitHub un token OIDC (`permissions: id-token: write`) e lo scambia con credenziali temporanee assumendo un ruolo IAM creato dallo stack `persistent`. Si è confinato tale ruolo su due fronti. Da una parte, la policy concede solo le azioni ECR necessarie al push ed esclusivamente sui tre repository del progetto (fa eccezione `ecr:GetAuthorizationToken`, che AWS non permette di limitare per risorsa). Dall'altra, la *trust policy* accetta esclusivamente token emessi per il branch `main` del repository in uso:

Codice 4: La condizione della trust policy che vincola il ruolo ai soli push su `main` (estratto da `terraform/aws/persistent/github_oidc.tf`).

```
condition {
  test      = "StringLike"
  variable = "token.actions.githubusercontent.com:sub"
  values = [
    "repo:${var.github_repo}:ref:refs/heads/main",
    # stesso vincolo, nella forma con gli ID numerici usata da GitHub
    "repo:${split("/", var.github_repo)[0]}@*/${split("/", var.github_repo)[1]}@*:ref:refs/heads/main",
  ]
}
```

Di conseguenza, operazioni provenienti da *pull request*, *fork* o repository esterni non sono autorizzate ad assumere tale ruolo. L'unico dato configurato nei *secret* del repository è l'ARN del ruolo stesso, un'informazione intrinsecamente non sensibile.

Il processo di *deploy* sul cluster è stato volutamente escluso dalla pipeline. Tale scelta architetturale è dettata dalla natura effimera del cluster, che viene distrutto al termine di ogni

sessione di lavoro. Un *job* di *rollout* automatico ad ogni *commit* andrebbe sistematicamente in errore in assenza dell'infrastruttura sottostante. Inoltre, fornire alla CI l'accesso diretto ad EKS ampliamente drasticamente la superficie di attacco del ruolo OIDC, violando il principio del minimo privilegio. L'applicazione dei manifest resta pertanto delegata al *playbook* Ansible (sezione 5.9), che viene lanciato solo quando l'ambiente è effettivamente operativo. Eseguendo il *playbook* con l'opzione `-e build_images=false`, esso scaricherà direttamente le immagini pre-buildate dalla CI. In uno scenario di produzione con cluster persistente, la transizione naturale per questo flusso sarebbe l'adozione di un approccio *GitOps pull-based* (es. ArgoCD o Flux), demandando al cluster stesso il compito di riconciliarsi con il repository.

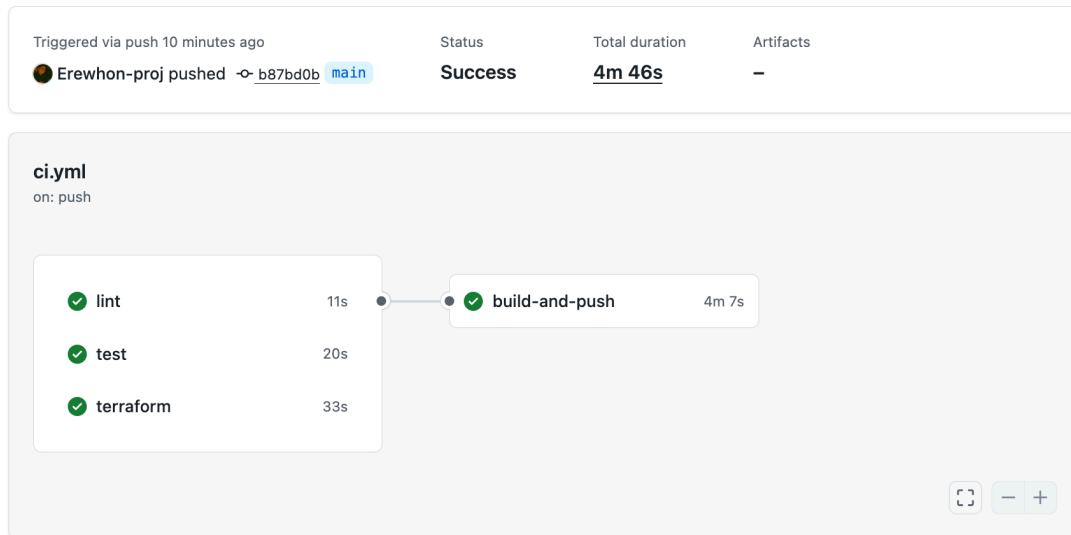


Figura 6: Una run della pipeline su GitHub Actions conclusa con successo: i job `lint`, `test` e `terraform` devono passare prima che `build-and-push` pubblichi le immagini, come mostrano le dipendenze tra i nodi.

7 Test e risultati

7.1 Metodologia

Le proprietà da verificare derivano direttamente dalle scelte architetturali, e per ciascuna si è definito un esperimento che si possa ripetere. Il carico è generato da `scripts/load_test.py`, che invia N recensioni, campionate da un insieme di testi realistici in italiano su banche diverse, in parallelo con un numero configurabile di connessioni concorrenti, misurando infine il tempo totale e il tasso di accettazione. Di seguito un esempio di avvio:

```
python scripts/load_test.py --url http://<API>:8000 --n 300 --concurrency 30
```

Sono stati previsti due esperimenti. Il primo verifica il disaccoppiamento dell'ingestione. Sotto un burst di 300 richieste concorrenti, tutte le risposte devono essere 202 e il throughput di accettazione deve restare dell'ordine delle decine di richieste al secondo, a prescindere dalla velocità di inferenza. È la controprova che l'API non sia accoppiata al modello; le metriche osservate sono le richieste al secondo riportate dallo script e il tasso di errore. Il secondo e ultimo esperimento riguarda l'autoscaling guidato dalla coda. Durante lo stesso burst si osserva la profondità della coda (l'interfaccia di management di RabbitMQ in Fase A, e `ApproximateNumberOfMessages` via CLI in Fase B) insieme al numero di pod worker (`kubectl get pods -w`). Ci si attende una partenza da 0 repliche, una salita proporzionale al backlog, circa una replica ogni 30 messaggi, fino a un massimo di 5 in locale e 4 in cloud, poi lo smaltimento e il rientro a 0 dopo il cooldown.

(30 s in locale, 300 s in cloud). In questa fase si misura il tempo di reazione dello scaler, il numero massimo di pod raggiunto, il tempo di svuotamento della coda e il tempo di rientro a zero.

7.2 Risultati

È stata eseguita la campagna quantitativa in Fase B (AWS, MODEL_MODE=pyabsa, 3 nodi, massimo 4 worker), campionando ogni 10 s lo stato delle recensioni nel database, la coda SQS e le repliche del worker (`kubectl get deploy worker`). La tabella 5 riporta i valori misurati, e la figura 7 mostra la curva completa. La Fase A, sviluppata per prima e su hardware locale con carico ridotto, è stata validata solo qualitativamente durante lo sviluppo, osservando direttamente `kubectl get pods -w`, senza una campagna numerica dedicata: il meccanismo (KEDA su lunghezza coda, scale $0 \rightarrow N \rightarrow 0$) è lo stesso, cambiano solo i tempi assoluti per via del `cooldownPeriod` più corto (30 s contro 300 s in cloud). A titolo indicativo, con il modello reale l'inferenza di una singola recensione richiede circa 4 secondi su CPU (MacBook M1, worker in container su OrbStack): è questa latenza a rendere indispensabili l'elaborazione asincrona e lo scaling orizzontale dei worker.

Tabella 5: Ingestione e scaling sotto burst da 300 richieste, concorrenza 30 (Fase B, AWS).

Metrica	Fase B (AWS)
Richieste accettate (202)	300/300
Throughput di ingestione (req/s)	58
Picco di profondità della coda	300 messaggi
Numero massimo di pod worker (desiderati / pronti)	4 / 4
Avvio dei worker, da richiesti a pronti (s)	~35
Tempo di svuotamento della coda dall'inizio del burst (s)	372 (~6 min 12 s)
Rientro a 0 repliche dopo l'ultima recensione completata (s)	301 (~300 + 1 s)

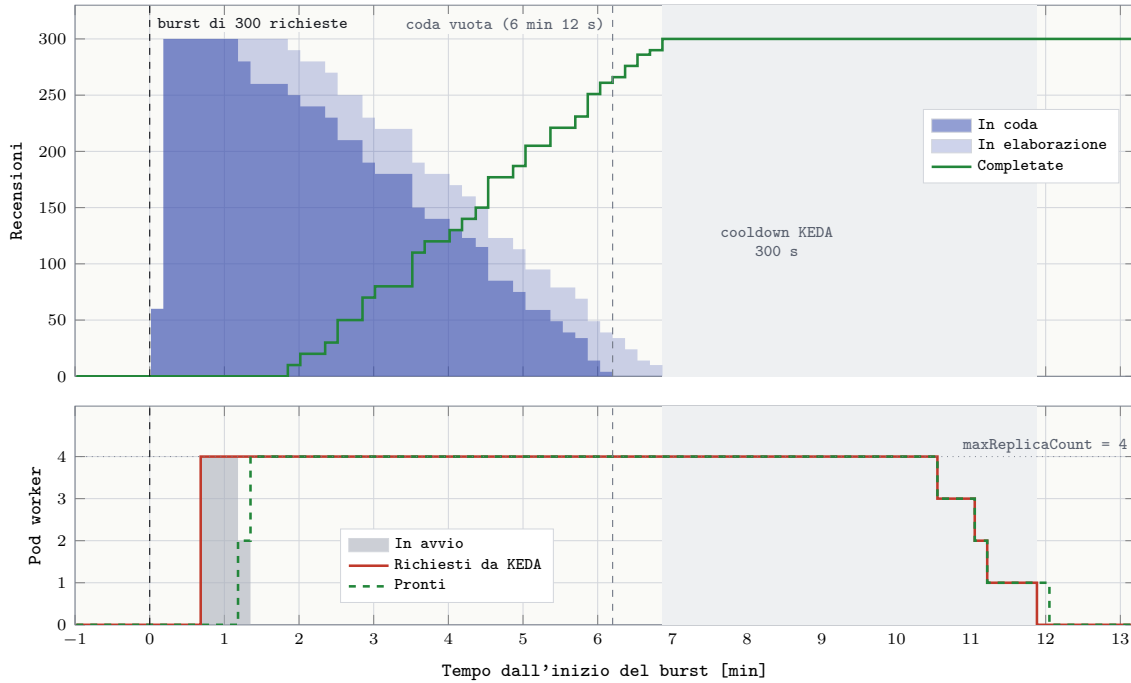


Figura 7: Dinamica dell'autoscaling durante il load test in Fase B (AWS), campionata ogni 10 s. In alto le recensioni in coda, in elaborazione e completate; in basso i pod worker richiesti da KEDA e quelli pronti. Si osservano la salita $0 \rightarrow 4$, lo svuotamento della coda e il rientro a 0 repliche dopo il `cooldownPeriod`.

Il comportamento qualitativo osservato in Fase A durante lo sviluppo (burst assorbito in pochi secondi, coda che cresce, scaling dei worker proporzionale al backlog, rientro a zero dopo il cooldown) coincide con quello quantificato in Fase B e riportato sopra: cambiano solo i tempi assoluti, più brevi in locale per via del `cooldownPeriod` ridotto.

8 Howto: riprodurre l'infrastruttura

Questa sezione è una guida operativa completa, dalla macchina vuota fino al teardown. I comandi si intendono eseguiti dalla radice del repository `absa-cloud`, salvo diversa indicazione.

8.1 Prerequisiti

- **Strumenti:** Docker (su macOS: OrbStack), `kubectl`, Terraform, Ansible, Python 3.11, AWS CLI. Per la Fase A serve un Kubernetes locale attivo (in OrbStack: *Settings, Kubernetes, Enable*); il contesto `kubectl` si chiama `orbstack` di default (altrimenti si passa `-e kube_context=...` al playbook).
- **Account AWS** (solo Fase B) con credenziali configurate (`aws configure`); regione di riferimento `eu-west-1`.
- **Collection Ansible** per la Fase B, una tantum:

```
ansible-galaxy collection install -r ansible/aws/requirements.yml
```

- **Checkpoint del modello** (solo per la modalità `pyabsa`): la cartella `lcf_atepc_custom_dataset_cdw_apcacc_89.41_apcf1_69.72_atef1_69.94` da scaricare in `models/` seguendo le istruzioni di `models/README.md`.

8.2 Fase A (locale)

Percorso rapido: Docker Compose.

```
docker compose up --build
# frontend http://localhost:8501
# API/docs http://localhost:8000/docs
# RabbitMQ http://localhost:15672 (guest/guest)
```

Per più worker: `docker compose up --build --scale worker=3 -d`. Per il modello reale si aggiunge il file `docker-compose.pyabsa.yml`, che builda il worker con `INSTALL_ML=true`, imposta `MODEL_MODE=pyabsa` e monta la cartella del checkpoint:

```
docker compose -f docker-compose.yml -f docker-compose.pyabsa.yml up --build
```

Percorso completo: Kubernetes + KEDA via IaC. Un solo comando esegue Terraform, builda le immagini, installa KEDA e applica i manifest:

```
ansible-playbook ansible/local/playbook.yml
# con il modello reale (applica k8s/local-pyabsa):
ansible-playbook ansible/local/playbook.yml -e model_mode=pyabsa
```

Verifica:

```
kubectl get pods -n absa-cloud          # api/frontend Running, worker a 0 repliche
kubectl get scaledobject -n absa-cloud  # worker-keda-scaler pronto
python scripts/load_test.py --url http://localhost:8000 --n 300 --concurrency 30
kubectl get pods -n absa-cloud -w      # KEDA scala il worker 0 -> N -> 0
```

8.3 Fase B (AWS)

1. Stack persistent (una tantum).

```
cd terraform/aws/persistent
cp terraform.tfvars.example terraform.tfvars
# impostare: model_bucket_name (univoco globalmente), email per i budget,
# github_repo (owner/repo) per il ruolo OIDC della CI
terraform init
terraform apply
```

Tra gli output c'è `github_actions_role_arn`: va copiato nel secret `AWS_ECR_PUSH_ROLE_ARN` del repository GitHub per abilitare la pipeline.

2. Stack compute (a ogni sessione).

```
cd ../compute
cp terraform.tfvars.example terraform.tfvars # stesso model_bucket_name
export TF_VAR_db_password='...' # 8-128 caratteri in [A-Za-z0-9_~]
terraform init
terraform apply # ~15-20 minuti (creazione EKS)
```

Se al primo `apply` il provider Helm fallisce perché il cluster non è ancora raggiungibile, un secondo `terraform apply` completa l'installazione di KEDA.

3. Deploy applicativo (Ansible).

```
cd ../../.. # radice del repository
ansible-playbook ansible/aws/playbook.yml # mock mode
ansible-playbook ansible/aws/playbook.yml -e upload_model=true # + modello su S3
# se la CI ha già pubblicato le immagini :latest su ECR:
ansible-playbook ansible/aws/playbook.yml -e build_images=false
```

Per l'inferenza reale, `MODEL_MODE` è già `pyabsa` nell'overlay `k8s/aws/01-config.yaml`; il caricamento del checkpoint su S3 (`-e upload_model=true`) serve solo la prima volta, perché il bucket è persistente.

4. Verifica end-to-end.

```
kubectl get pods -n absa-cloud           # api/frontend Running, worker 0 repliche
kubectl get scaledobject -n absa-cloud    # worker-sqs-scaler attivo

API=$(kubectl get svc api -n absa-cloud \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
FRONTEND=$(kubectl get svc frontend -n absa-cloud \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')

python scripts/load_test.py --url "http://$API:8000" --n 300 --concurrency 30
kubectl get pods -n absa-cloud -w        # scaling 0 -> N -> 0
SQS_URL=$(terraform -chdir=terraform/aws/compute output -raw sqs_queue_url)
aws sqs get-queue-attributes --queue-url "$SQS_URL" \
  --attribute-names ApproximateNumberOfMessages
```

Il frontend è raggiungibile su `http://$FRONTEND`.

8.4 Teardown

A fine sessione viene distrutto solo lo stack `compute`: EKS, RDS, NAT, load balancer e SQS spariscono e la spesa oraria si azzerà; S3 (con il modello), ECR (con le immagini) e i budget restano attivi.

```
./scripts/aws_teardown.sh
```

Lo script verifica che il contesto `kubectl` sia quello di EKS, elimina il namespace (e con esso i load balancer creati da Kubernetes, che altrimenti bloccherebbero la distruzione della VPC) e poi esegue `terraform destroy` sullo stack `compute`. Per riprendere il lavoro basta un `terraform apply` sullo stack `compute` e un nuovo giro di `ansible-playbook`; il modello su S3 non va ricaricato. In Fase A l'equivalente è `terraform destroy` in `terraform/local` (o `docker compose down -v`), che non ha comunque nessuna implicazione di costo.

9 Il repository

Il codice è stato organizzato in un unico repository, **absa-cloud**, con una struttura che separa i servizi applicativi dall'infrastruttura:

```
absa-cloud/
|- services/
|  |- api/          # FastAPI gateway (app/main.py, queue.py, db.py)
|  |- worker/       # consumer + inferenza (app/worker.py, inference.py)
|  |  |- requirements-ml.txt # dipendenze PyABSA/torch (solo INSTALL_ML=true)
|  |- frontend/     # Streamlit (app.py)
|- db/init.sql      # schema PostgreSQL (idempotente)
|- models/          # checkpoint del modello (fuori da git, vedi README.md)
|- k8s/
|  |- local/        # manifest Fase A (Postgres, RabbitMQ, app, KEDA)
|  |- local-pyabsa/ # variante locale con il modello reale (kustomize)
|  |- aws/          # overlay Fase B (template con placeholder ${...})
|- terraform/
|  |- local/        # namespace sul cluster OrbStack
|  |- aws/persistent/ # S3, ECR, Budgets, ruolo OIDC per la CI
|  |- aws/compute/   # VPC, EKS, RDS, SQS, IRSA, KEDA
|- ansible/
|  |- local/playbook.yml # provisioning + build + deploy Fase A
|  |- aws/playbook.yml   # deploy Fase B (legge gli output Terraform)
|- scripts/
|  |- load_test.py       # generatore di carico
|  |- render_manifest.py # sostituzione dei placeholder nei manifest
|  |- aws_teardown.sh    # distruzione dello stack compute
|- tests/               # test unitari (worker, API, mock, render dei manifest)
|- .github/workflows/ci.yml # lint, test e terraform, poi push su ECR
|- docker-compose.yml   # ambiente di sviluppo rapido
|- docker-compose.pyabsa.yml # variante Compose con il modello reale
|- requirements-dev.txt  # dipendenze per test e lint
|- ruff.toml            # configurazione del linter
|- docs/               # schema, slide e relazione
```

Ci sono alcune convenzioni che si è cercato di rispettare in tutto il repository. Ogni servizio è autocontenuto: Dockerfile, requirements e codice stanno nella stessa cartella, e nessun servizio importa codice da un altro. La configurazione è solo ambientale, mai scritta a mano nel codice; i valori di default sono quelli della Fase A, così l'ambiente locale funziona senza dover configurare nulla in anticipo. I manifest Kubernetes sono numerati nell'ordine in cui vanno applicati. I manifest AWS sono template: gli unici segnaposto hanno la forma `${VAR}` e vengono risolti a deploy-time dagli output di Terraform, quindi nel repository non compaiono ARN, endpoint o URL legati a un account specifico. Gli stati Terraform e i `tfvars` con valori reali restano fuori dal controllo di versione.

10 Conclusioni

A partire da un'applicazione a tre livelli composta da microservizi containerizzati, la si è fatta eseguire su un ambiente di sviluppo locale e poi è stata migrata su un deployment cloud su servizi gestiti, senza toccare il codice applicativo. Le differenze tra le due fasi vivono interamente nella configurazione e nell'infrastruttura, che a sua volta è descritta come codice. La scelta architetturale su cui si è puntato di più, l'inferenza asincrona dietro una coda con autoscaling event-driven e scale-to-zero, si è rivelata la risposta giusta al vincolo più severo del lavoro, cioè servire un modello BERT usando solo CPU. La coda assorbe i picchi, lo scaling orizzontale dei

worker compensa l'assenza di GPU e KEDA adegua la capacità al backlog effettivo, spegnendo tutto quando non serve.

Il limite che ha pesato di più è stato il budget, e molte delle scelte descritte nei capitoli precedenti ne discendono direttamente. Si è usato un solo NAT Gateway invece di uno per zona, il che lascia le subnet private di due zone su tre dipendenti da un unico punto di uscita verso Internet. I nodi sono istanze spot senza GPU, quindi l'inferenza gira su CPU e un batch costa secondi invece che millisecondi. Lo stack è diviso in una parte persistente e una effimera per poter distruggere il cluster a fine sessione, e RDS resta single-AZ. Va detto però che questi vincoli si sono rivelati buone pratiche più che semplici ripieghi, perché il teardown è diventato un'operazione a un comando, sicura e ripetibile, e le istanze spot sono sostenibili proprio perché l'architettura tollera per costruzione la perdita di un worker.

Restano poi alcuni limiti coerenti con la natura didattica del lavoro. Non c'è uno stack di osservabilità vero e proprio, perché il monitoraggio si affida a `kubect1`, ai log e alle console dei servizi, mentre un deployment di produzione richiederebbe metriche e dashboard dedicate, per esempio Prometheus/Grafana o CloudWatch Container Insights. Il traffico viaggia in HTTP semplice sugli endpoint dei load balancer, senza TLS né autenticazione dell'API, e i segreti applicativi passano per Secret Kubernetes ordinari invece che per un secret manager dedicato. Infine non c'è un Cluster Autoscaler, quindi il numero di nodi EKS è fisso e il tetto dei worker (`maxReplicaCount`) va scelto a mano in base alla capacità disponibile. Nessuno di questi punti tocca l'architettura di fondo, sono estensioni che si potrebbero aggiungere in un secondo momento senza stravolgere nulla.

Riferimenti bibliografici

- [1] M. Pontiki, D. Galanis, J. Pavlopoulos, H. Papageorgiou, I. Androutsopoulos, S. Manandhar, *SemEval-2014 Task 4: Aspect Based Sentiment Analysis*, Proceedings of SemEval 2014, pp. 27–35, ACL, 2014. <https://aclanthology.org/S14-2004/>
- [2] H. Yang, B. Zeng, J. Yang, Y. Song, R. Xu, *A Multi-task Learning Model for Chinese-oriented Aspect Polarity Classification and Aspect Term Extraction*, arXiv:1912.07976, 2019. <https://arxiv.org/abs/1912.07976>
- [3] H. Yang, C. Zhang, K. Li, *PyABSA: A Modularized Framework for Reproducible Aspect-based Sentiment Analysis*, arXiv:2208.01368, 2022. <https://arxiv.org/abs/2208.01368>
- [4] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Proceedings of NAACL-HLT 2019, pp. 4171–4186. <https://aclanthology.org/N19-1423/>
- [5] MDZ Digital Library team, *Italian BERT (dbmdz/bert-base-italian-xxl-cased)*, Bayerische Staatsbibliothek. <https://huggingface.co/dbmdz/bert-base-italian-xxl-cased>
- [6] G. Hohpe, B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*, Addison-Wesley, 2003. <https://www.enterpriseintegrationpatterns.com/>
- [7] J. Lewis, M. Fowler, *Microservices: a definition of this new architectural term*, 2014. <https://martinfowler.com/articles/microservices.html>
- [8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, *Borg, Omega, and Kubernetes*, ACM Queue, vol. 14, n. 1, 2016. <https://queue.acm.org/detail.cfm?id=2898444>
- [9] The Kubernetes Authors, *Kubernetes Documentation*. <https://kubernetes.io/docs/home/>
- [10] KEDA Authors, *KEDA — Kubernetes Event-driven Autoscaling, Documentation*. <https://keda.sh/docs/>
- [11] HashiCorp, *Terraform Documentation*. <https://developer.hashicorp.com/terraform/docs>
- [12] Amazon Web Services, *Amazon EKS Best Practices Guide*. <https://docs.aws.amazon.com/eks/latest/best-practices/introduction.html>
- [13] Amazon Web Services, *IAM roles for service accounts*, Amazon EKS User Guide. <https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>

Elenco delle figure

1	Architettura della Fase A (locale)	6
2	Sequenza di elaborazione di una recensione	7
3	Dashboard del sentiment in Fase A	11
4	Architettura della Fase B (AWS)	12
5	Topologia di rete della Fase B	13
6	Pipeline CI/CD	18
7	Dinamica dell'autoscaling (Fase B)	19

Elenco delle tabelle

1	Componenti della piattaforma	5
2	Variabili d'ambiente	9
3	Servizi AWS impiegati	12
4	Costi orari effettivi (stack compute)	16
5	Ingestione e scaling sotto burst	19